

AI VERIFICATION & VALIDATION LAB — INTERNSHIP SUMMARY

From Backend Engineer to Closed-Loop V&V Research

Learning, building, and evaluating decision-making policies under uncertainty

Ahmad Alfian Alfian Irfan

AI VnV Research Intern

THE STARTING POINT

A backend engineer, curious about uncertainty

- Background: backend engineering — not computer-science research
- The curiosity: how do agents **decide under uncertainty**? (POMDPs)
- Month one, honestly: unfamiliar ground — the theory, the tooling, the papers
- The result was **bad performance** — and a decision to fix it properly

The gap was fundamentals, not effort.

THE RESET

Back to fundamentals, deliberately

- Diagnosed the gap: decision-making theory and its ecosystem
- **Coursera** — decision-making course, from MDP basics up
- **JuliaAcademy** — Decision Making Under Uncertainty with POMDPs.jl
- Rule adopted: every concept gets **rebuilt in code** before moving on

Learn → build → verify became the working loop for everything after.

Pluto notebooks as the learning medium

- Hands-on **Pluto notebooks**: MDP and POMDP from first principles
- Interactive **solver playground** — explore the JuliaPOMDP solver ecosystem side by side
- Worked through the **Gallery of POMDPs.jl** examples
- The notebooks double as a learning medium for the next intern

github.com/PannnTastic/Decision-Making-Under-Uncertainty

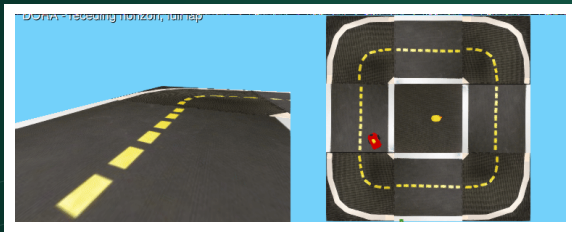
Certified explanations for MDP driving policies



- MDP formulation of Duckietown driving; explanations in three pillars: **why this action** · **what if another** · **is it consistent**
- Counterfactual rollouts share exogenous noise — outcome claims are settled **exactly**, across Q-learning, SARSA, SAC, TD3

github.com/PannnTastic/DuckieMDP · arXiv preprint

The simulator, reborn in Julia



DORA solver completing a lap, drawn by the package's native renderer

- Native **POMDPs.jl** reimplementation of the DuckieMDP environment
- Validated **decision by decision** against Python — exact NumPy RNG streams, seeded episodes reproduce **bit for bit**
- Native renderer; DORASolvers.jl case study drives a full lap

github.com/ai-vnv/Duckietown.jl

MAIN PROJECT — SUBMITTED TO IEEE OJ-ITS

A **Closed-Loop** Evaluation of Capability **Loss and Recovery** in Compressed Driving Policies

From learning POMDPs to using one as a research instrument.

THE PROBLEM

A policy can pass the benchmark and fail to stop

- Learned driving policies run on embedded computers
- Standard practice: prune, distill, quantize before deployment
- Progress measured by size, latency, aggregate accuracy
- Aggregate scores hide **which behaviors** compression removes

The question is not only how small the network can get — but which learned capabilities survive each stage.

KEY IDEA

Closed loop, stage by stage — reversed

- A policy acts on its own observations: small action drift **compounds** through the feedback loop
- Static test-set scores cannot see this
- Scenario-based assessment, reversed: **fix the scenarios and seeds**, vary the system under test
- Every compression stage is a new configuration to re-certify; acceptance criteria fixed **before** any result

Four questions: where does driving break · is it recoverable · does the data decide · does recovery survive lower precision.

THE DRIVING TASK

Five curricula, shared seeds, rising difficulty



C0 lane following



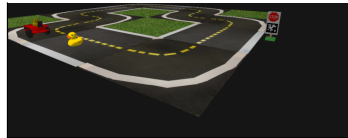
C1 larger loop



C2 crossing pedestrian



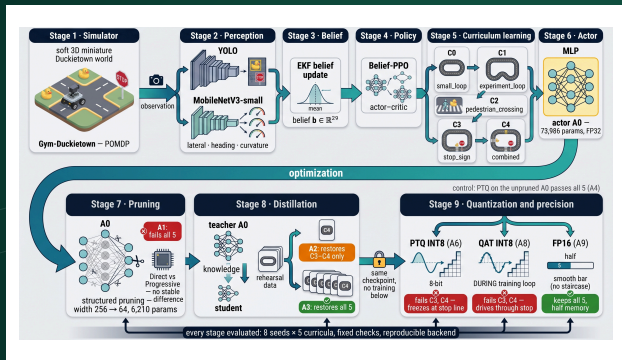
C3 stop compliance



C4 combined

- C0–C1 lane following · C2 crossing pedestrian · C3 stop sign · C4 both
- The stop curricula demand interrupting the drive, then re-establishing it

A belief-state visuomotor policy (POMDP + PPO)



- Camera → **YOLO11n** (objects) + **MobileNetV3** (lane pose)
- EKFs fuse detections over time into a **29-d belief** vector
- Belief-PPO actor: **73,986** parameters (FP32)
- Only the actor is compressed; perception stays fixed

Ten configurations, one shared ancestry

- **A0** original actor (reference)
- **A1** structured pruning, hidden width 256 $\rightarrow$ 64
- **A2** + distillation on the hardest curriculum only
- **A3** + distillation on balanced C0–C4 data (62,176 states)
- **A6** A3 $\rightarrow$ post-training INT8 **A8** A3 parent $\rightarrow$ QAT INT8
- **A9** A3 $\rightarrow$ FP16 cast (precision control)
- **A4, A5, A7** controls: unpruned INT8, no-distill INT8, historical QAT

Every arrow is evaluated the same way — no configuration is trusted on the previous stage's evidence.

EVALUATION PROTOCOL

400 matched episodes, two independent axes

- **10 models** × **5 curricula** × **8 seeds** — identical seeds for all
- Acceptance checks preregistered: completion, progress, collisions, stop compliance and restart, lane keeping, clearance
- Evaluation backend verified **bit-for-bit** reproducible
- Axis 1: task verdicts (does it drive?)
- Axis 2: action fidelity vs. the original on identical inputs (does it imitate?)

Pruning breaks driving; rehearsal decides recovery

- Structured pruning: 73,986 $\rightarrow$ **6,210** parameters (-91.6%) — **fails all five** curricula
- Same teacher, loss, optimizer, budget — only the rehearsal data differs
- Distill on the hardest curriculum only $\rightarrow$ recovers **C3–C4 only**
- Balanced rehearsal across all five $\rightarrow$ recovers **all five**

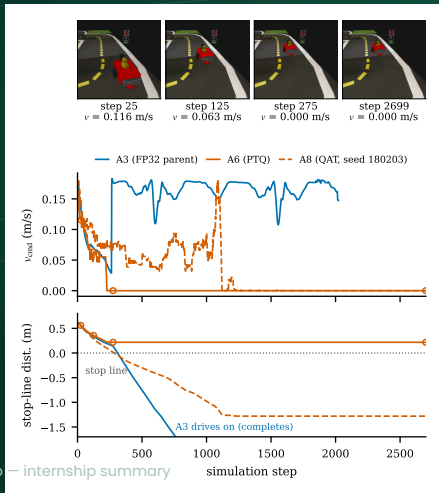
What the student rehearses is what the student keeps.

INT8 undoes a completed recovery

A1 prune	fail	fail	fail	fail	fail
A2 prune+KD(C4)	fail	fail	fail	pass	pass
A3 prune+KD(bal)	pass	pass	pass	pass	pass
A4 PTQ only	pass	pass	pass	pass	pass
A5 prune+PTQ	fail	fail	fail	fail	fail
A6 KD(bal)+PTQ	pass	pass	pass	fail	fail
A7 KD(C4)+PTQ+QAT	fail	fail	fail	pass	pass
A8 KD(bal)+QAT	pass	pass	pass	fail	fail
A9 KD(bal)+FP16	pass	pass	pass	pass	pass
	C0	C1	C2	C3	C4

- Both INT8 routes fail C3–C4 from one byte-identical checkpoint
- Stop-task completion falls from **8/8** to **3/8** under PTQ

One checkpoint, two ways to fail a stop



- **A6 (PTQ)** brakes correctly, parks **0.22 m** before the line — and never moves again (to the 2700-step horizon)
- **A8 (QAT)** drives **through** the stop at low speed while the stop is still required
- Same parent, same task, opposite failures

FINDINGS 4 & 5

Controls and fidelity: the cause is the interaction

- Same PTQ on the **unpruned** actor → passes all five
- **FP16 cast** of the recovered actor → keeps all five
- So the failure is **narrow width** × **INT8**, not precision alone
- QAT imitates the original **more closely** than PTQ — and drives worse
- The unpruned INT8 control drives everything — and fails the similarity thresholds

Action-level similarity is not an acceptance test — in either direction.

Actor cost across precisions

	Param. memory	Median latency	All curricula
A3 FP32	24.8 KB	19.4 μ s	pass
A9 FP16	12.4 KB	24.5 μ s (+26%)	pass
A6 INT8	7.6 KB	12.9 μs (1.50 $\times$)	fail C3–C4

- FP16 halves memory but is slower here (no native half path on this CPU)
- The actor is under **0.1%** of end-to-end step cost — perception dominates

TAKEAWAYS

Judge compressed policies by how they drive

- Compression is a sequence of **behavioral transitions**: lose, recover, lose again
- Evaluate **every stage** in closed loop — no stage is safe by inheritance
- Choose rehearsal data to cover **everything** the policy must keep
- A completed recovery is **not** robust to a later precision cut
- Action-level similarity is not an acceptance test

FUTURE WORKS

Where this goes next

- **Wider settings:** more quantization configurations, more training realizations, more seeds
- **Compress perception:** the other 99.9% of step cost, under the same acceptance protocol
- **Transfer the protocol:** other policy stacks, other simulators, eventually hardware
- **Julia-native line:** Duckietown.jl as the ground for solver-level V&V experiments

WHAT THE INTERNSHIP BUILT

Fundamentals first, then honest evaluation

- From backend engineering to an **RL / POMDP research workflow**
- **4 public repos · 2 papers · 1 Julia package**
- From a bad first month to an **OJ-ITS submission**
- The lesson that stuck: measure what matters, **in closed loop**

THANK YOU

Questions?

Everything in this talk is public:

github.com/ai-vnv/CompressedAutoDriving

github.com/PannnTastic/DuckieMDP

github.com/PannnTastic/Decision-Making-Under-Uncertainty

github.com/ai-vnv/Duckietown.jl

Internship summary — AI Verification & Validation Lab